DEPURACIÓN DE CÓDIGO

Entornos de Desarrollo

Marco Valiente
DAW1

Contenido

Introducción 2

Desarrollo 3

1. Selección de líneas a depurar 3
2. Inicio del Debug 5

Conclusiones 15

Introducción

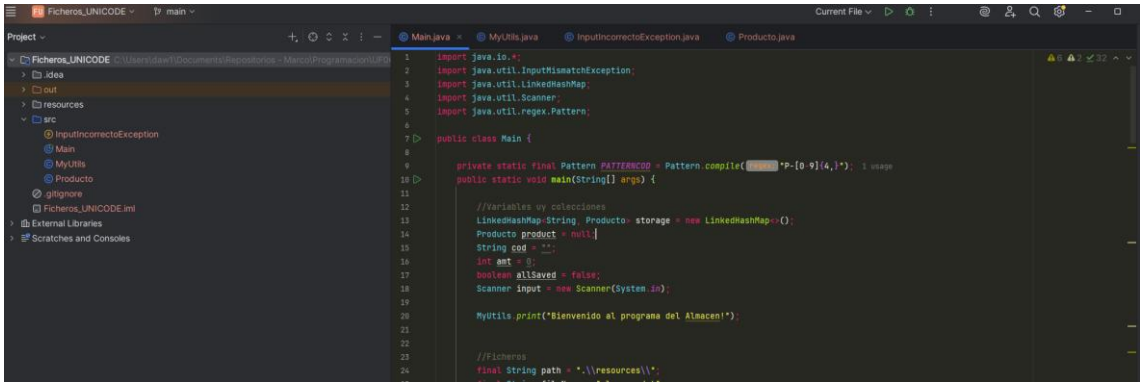
En este documento, vamos a realizar una actualización sobre cómo el código de nuestros programas se ejecuta, siguiendo a través de la herramienta de debuggear en IntelliJ cómo van cambiando nuestras variables.

Para esto, vamos a escoger el trabajo más reciente que nosotros tenemos, que es el de Ficheros UNICODE.

Desarrollo

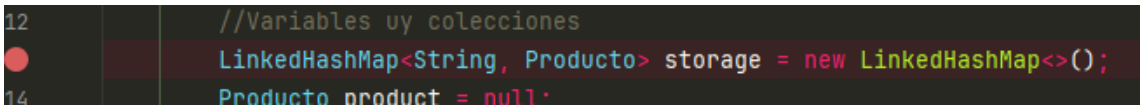
1. Selección de líneas a depurar

Empezamos abriendo nuestro programa en el IDE.



Y ahora, desde aquí, vamos a decidir dónde vamos a parar.

Para estas paradas, vamos a escoger las siguientes líneas de código:



Cómo se crea el LinkedHashMap

```java
try {
    almacenFile = new File( pathname: path + fileName);
    if (almacenFile.createNewFile()) {
        MyUtils.print("Creando el archivo almacen: " + fileName);
    } else {
        MyUtils.print("El archivo ya existe, hay datos que cargar");
    }
} catch (IOException e) {
    MyUtils.print("Error al crear el archivo almacen: " + e.getMessage());
}

try (FileReader fileReader = new FileReader(almacenFile);
    BufferedReader bufferedReader = new BufferedReader(fileReader)) {
    while (line != null) {
        line = bufferedReader.readLine();
        if (line != null && !line.equals("")) {
            lineData = line.split( regex: ",");
            String codigo = lineData[0];
            String nombre = lineData[1];
            int cantidad = Integer.parseInt(lineData[2].trim());
            double precio = Double.parseDouble(lineData[3].trim());

            storage.put(codigo, new Producto(codigo, nombre, cantidad, precio));
            MyUtils.print("Producto ha sido almacenado: " + codigo);
        }
    }
} catch (IOException e) {
    MyUtils.print("No se pudo usar el documento");
    MyUtils.print(e.getMessage());
    return;
} catch (InputMismatchException e) {
```

Cómo se abre y se leen los archivos que nuestro programa trabaja con.

También:

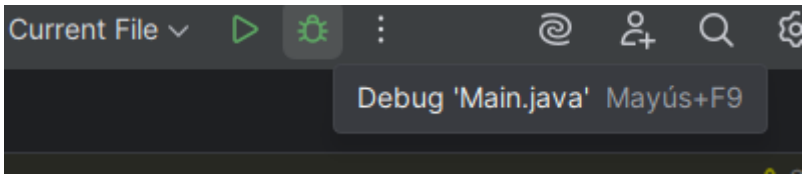
```java
public static Producto createProduct() {  1 usage
    Producto newProduct;
    String codigo, nombre;
    int cantidad = 0;
    double precio = 0.0;
```

Cómo se crea un producto incluso a medida que se le dan inputs incorrectos.

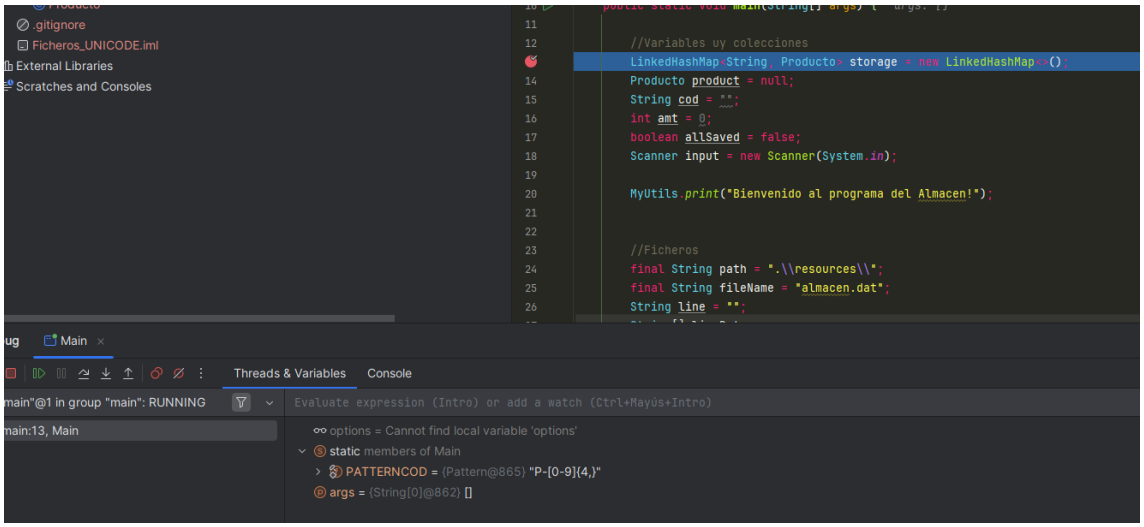
```java
    //Creacion Objeto
    newProduct = new Producto(codigo,nombre,cantidad,precio);

    return newProduct;
}
}
```

Parando muy al final.

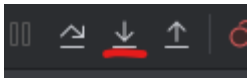
2. Inicio del Debug



Con los errores marcados, iniciamos el programa dándole click a Debug.

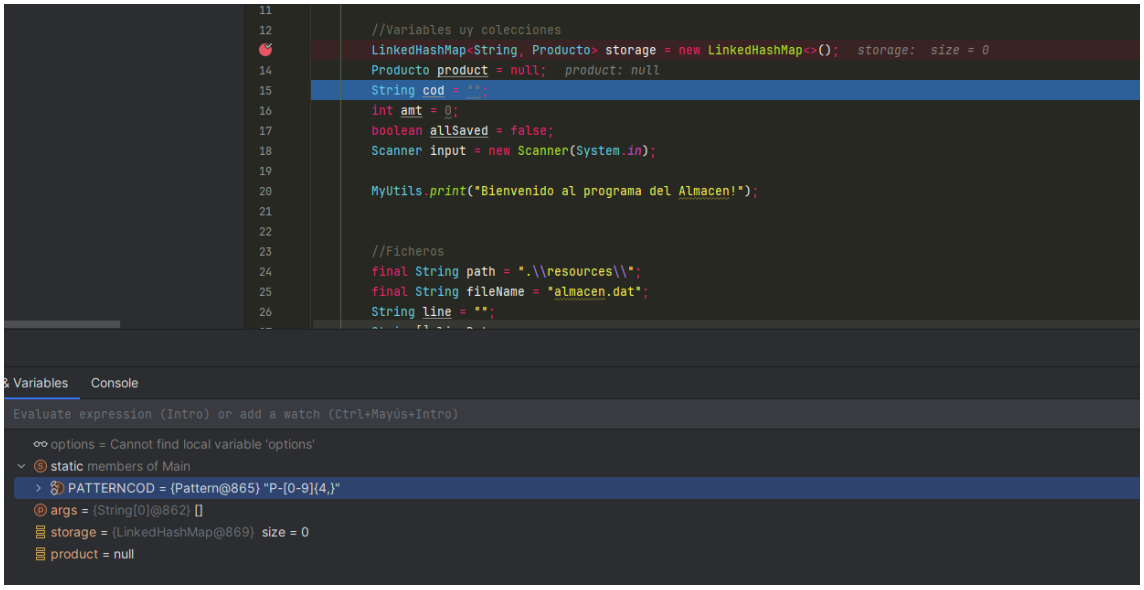


Le damos click, e inmediatamente paramos el programa en la 13ª línea. Podemos observar desde aquí que el Patrón y los argumentos del main son las únicas cosas que actualmente están ejecutadas.



Seleccionamos ahora Step Into, para ver dónde nos están metiendo.

Y… en ningún lado. Seguimos leyendo las líneas. Podemos seguir línea por línea.



Pero esto no nos interesa ya. Podemos usar Step Out para saltar a nuestro siguiente punto en rojo.

```
29
30      try {
31          almacenFile = new File( pathname: path + fileName);   path: ".\resources\"
32          if (almacenFile.createNewFile()) {
33              MyUtils.print("Creando el archivo almacen: " + fileName);   fileName: "almacen.dat"
34          } else {
35              MyUtils.print("El archivo ya existe, hay datos que cargar");
36          }
37      } catch (IOException e) {
38          MyUtils.print("Error al crear el archivo almacen: " + e.getMessage());
39      }
40
41      try (FileReader fileReader = new FileReader(almacenFile);   almacenFile: ".\resources\almacen.dat"
42          BufferedReader bufferedReader = new BufferedReader(fileReader)) {
```

Nos salta directamente hasta la línea 41, ya que, al parecer, no se llega a ejecutar el try de la línea 30. Podemos ver eso debido al símbolo de prohibido que se marca en nuestro lado izquierdo.

Aquí podemos seguir con Step Over para continuar con cómo se va leyendo el programa.

```
40
41      try (FileReader fileReader = new FileReader(almacenFile);   fileReader: FileReader@1156   almacenFile: ".\resources\almacen.d
42          BufferedReader bufferedReader = new BufferedReader(fileReader)) {   fileReader: FileReader@1156
43          while (line != null) {
44              line = bufferedReader.readLine();
45              if (line != null && !line.equals("")) {
46                  lineData = line.split( regex: ",");
47                  String codigo = lineData[0];
48                  String nombre = lineData[1];
49                  int cantidad = Integer.parseInt(lineData[2].trim());
50                  double precio = Double.parseDouble(lineData[3].trim());
51
52                  storage.put(codigo, new Producto(codigo, nombre, cantidad, precio));
53                  MyUtils.print("Producto ha sido almacenado: " + codigo);
54              }
55          }
56      } catch (IOException e) {

Variables   Console
Evaluate expression (Intro) or add a watch (Ctrl+MayÚs+Intro)
> line = ""
  almacenFile = {File@1121} ".\resources\almacen.dat"... Open
v fileReader = {FileReader@1156}
  > sd = {StreamDecoder@1158}
  v lock = {FileInputStream@1159}
    > fd = {FileDescriptor@1160}
    > path = ".\resources\almacen.dat"
      channel = null
    > closeLock = {Object@1162}
      closed = false
```

Seguimos leyendo, el FileReader ha sido creado de forma exitosa.

```
    }

    try (FileReader fileReader = new FileReader(almacenF
        BufferedReader bufferedReader = new BufferedRead
        while (line != null = true) {   line: ""
            line = bufferedReader.readLine();
```

Podemos ver que line es diferente a null, ya que esta tiene el contenido “”, lo que es considerado distinto.

Marcamos la línea con el punto para poder comprobar su valor en cada loop.

```
while (line != null = true) {   line: ""
    line = bufferedReader.readLine();
```

Y ahora, para no ir manualmente por cada objeto leído de nuestro archivo, podemos usar Step Out.

```
while (line != null = true) {   line: "P-0001,Play Station 5,2,465434.54"
    line = bufferedReader.readLine();
    if (line != null && !line.equals("")) {
        lineData = line.split( regex: ",");
        String codigo = lineData[0];
        String nombre = lineData[1];
        int cantidad = Integer.parseInt(lineData[2].trim());
        double precio = Double.parseDouble(lineData[3].trim());

        storage.put(codigo, new Producto(codigo, nombre, cantidad, precio));
        MyUtils.print("Producto ha sido almacenado: " + codigo);
    }
```

Primera ejecución. Se ha leído la siguiente línea, del objeto P-0001 que tenemos.

Lo mismo ocurre con todos los demás objetos.

```
while (line != null = true) {   line: "P-0003,XBox 3,43,9000.0"
    line = bufferedReader.readLine();
```

```
while (line != null = true) {   line: "P-0004,Palo de Pegar,32,1223.43"
```

```
BufferedReader bufferedReader = new BufferedReader(fileReader)) {   bufferedReader: BufferedReade
while (line != null = true) {   line: "P-0005,Libros del Senior de los Anillos,12,200.0"
    line = bufferedReader.readLine();
```

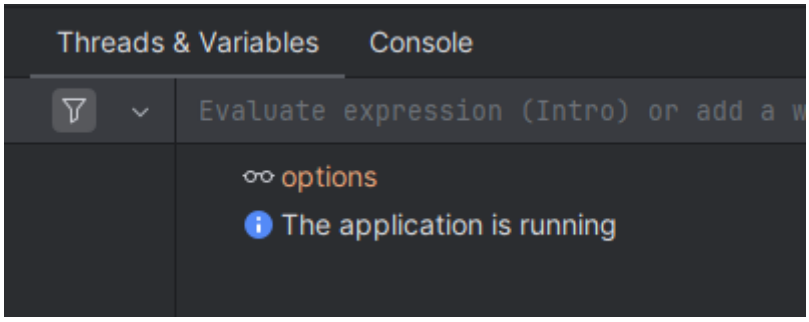
```
while (line != null = true) {   line: "P-0006,Marionetas,24,10.11"
```

```
while (line != null = true) {   line: "P-0007,Macarrones,60,5.5"
```

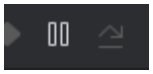
```
while (line != null = false) {   line: null
    line = bufferedReader.readLine();
    if (line != null && !line.equals("")) {
        lineData = line.split( regex: ",");
        String codigo = lineData[0];
        String nombre = lineData[1];
        int cantidad = Integer.parseInt(lineData[2].trim());
        double precio = Double.parseDouble(lineData[3].trim());

        storage.put(codigo, new Producto(codigo, nombre, cantidad, pre
        MyUtils.print("Producto ha sido almacenado: " + codigo);
    }
}
} catch (IOException e) {
    MyUtils.print("No se pudo usar el documento");
    MyUtils.print(e.getMessage());
    return;
```

Finalmente, ha llegado a null. No hay más documento a leer. El código se nos marca en gris.

Le damos a Step Over, y ahora vamos a tener que ir a seleccionar a crear objeto.



Nuestras variables se nos marcan tal que así ya que el programa no está parado ya.



Aunque podemos usar el botón de pausa para comprobar dichos pasos.

```
Datos del Almacen
1. Crear producto
2. Mostrar productos existentes
3. Eliminar producto por codigo
4. Restaurar ultimo cambio del inventario
5. Modificar cantidad de un producto
6. Guardar productos en el Fichero
7. Salir

Introduce el numero de una opcion: 1
```

Y aquí, introducimos la opción 1. Crear producto, para ver cómo se va creando.

```
251
252     public static Producto createProduct() {
            Producto newProduct;
254         String codigo, nombre;
```

La primera pausa no ocurre. El Producto no se inicializa, por lo que no nos sirve de nada parar ahí. No hay código que se ejecute que produzca que se pare.

```
Introduce el numero de una opcion: 1

Creando un nuevo producto:

Introduce el codigo del producto:
```

Seguiremos escribiendo en nuestro archivo como llevábamos haciendo.

```
Introduce el codigo del producto: P-0008

Introduce el nombre del producto: Espada

Introduce la cantidad del producto: 32

Introduce el precio del producto de hasta 2 decimales: 112,99
```

Le introducimos los siguientes datos, y el programa se pausa.

```
288     //Creacion Objeto
        newProduct = new Producto(codigo,nombre,cantidad,precio);   cantidad: 32   precio: 112.99   codigo: "P-0008"   nombre: "Espada"
```

Ha llegado a la línea de construcción del objeto.

```
options = Cannot find local variable 'options'
> static members of Main
  cantidad = 32
  precio = 112.99
> in = {Scanner@1247} "java.util.Scanner[delimiter
> codigo = "P-0008"
> nombre = "Espada"
  success = true
```

Vemos las variables activas. Todas coinciden. Podemos ahora seguir con Step Into y ver el constructor.

```
    public Producto(String codigo, String nombre, int cantidad, double precio) {  2 usages
        this.nombre = nombre;
        this.cantidad = cantidad;
        this.precio = precio;
        this.codigo = codigo;
    }
```

Y desde aquí seguimos con Step Over, viendo cómo se construye.

```
20        this.nombre = nombre;   nombre: "Espada"    nombre: null
21        this.cantidad = cantidad;
22        this.precio = precio;
23        this.codigo = codigo;
24    }
25
26    //Getter y Setter
27    public String getCodigo() {  2 usages
28        return codigo;
29    }
```

```
ariables   Console
aluate expression (Intro) or add a watch (Ctrl+Mayús+Intro)
options = Cannot find local variable 'options'
this = {Producto@1253} Object is being initialized
codigo = "P-0008"
nombre = "Espada"
cantidad = 32
precio = 112.99
this.precio = 0.0
this.cantidad = 0
this.nombre = null
```

Sobrescribiendo los valores que anteriormente eran null.

```
public Producto(String codigo, String nombre, int cantidad, double precio) {  2 usages
    this.nombre = nombre;   nombre: "Espada"    nombre: "Espada"
    this.cantidad = cantidad;   cantidad: 32    cantidad: 32
    this.precio = precio;   precio: 112.99    precio: 112.99
    this.codigo = codigo;   codigo: "P-0008"    codigo: null
}
```

Así seguimos, elemento por elemento.

```
288
289         //Creacion Objeto
            newProduct = new Producto(codigo,nombre,cantidad,precio);
291
292         return newProduct;
293     }
294 }
```

Se nos devuelve a la línea con comentario, después de esto.

```
92          switch (selected) {   selected: '1' 49
93              case '1':
94                  MyUtils.print("\nCreando un nuevo producto:");
95                  product = createProduct();
96                  product = storage.put(product.getCodigo(), product);
97                  if (product == null) {
98                      MyUtils.print("\nProducto creado exitosamente");
99                  }
100                 else {
101                     MyUtils.print("\nProducto creado exitosamente");
102                     MyUtils.print("\nEl siguiente producto ha sido sobreescrito:");
103                     MyUtils.print(product.toString());
104                 }
105                 allSaved = false;
106                 MyUtils.pause();
107                 break;
108             case '2':
109                 MyUtils.print("\nMostrando todos los productos:\n");
```

```
Variables   Console
Evaluate expression (Intro) or add a watch (Ctrl+Mayús+Intro)
fileName = "almacen.dat"
line = null
almacenFile = {File@1121} ".\resources\almacen.dat" ... Open
menu_options = {Pattern@1258} "[1-7]"
title = "Datos del Almacen"
options = {String[7]@1260} ["Crear producto", "Mostrar product...", "Eliminar produc...", "Restaurar ultim...", "Modificar canti...", +2 more]... Explore Elements
inputPetition = "Introduce el numero de una opcion: "
selected = '1' 49
```

Y estamos de vuelta en el método main, fuera de este método auxiliar crearProducto().

```
switch (selected) {   selected: '1' 49
    case '1':
        MyUtils.print("\nCreando un nuevo producto:");
        product = createProduct();   product: "Producto{codigo='P-0008', nombre='Espada', cantidad=32, precio=112.99}"
        product = storage.put(product.getCodigo(), product);
```

Vemos que nuestro objeto está introducido correctamente.

```
        case '1':
            MyUtils.print("\nCreando un nuevo producto:");
            product = createProduct();
            product = storage.put(product [NullPointerException].getCodigo(), product);   product: null    storage:  size = 7
            if (product == null) {
                MyUtils.print("\nProducto creado exitosamente");
```

Y ahora se inserta dentro de nuestro almacén de datos. Podemos ver que ahora nos devuelve null, ya que un LinkedHashMap devuelve la casilla que la Key reemplaza, que en este caso no existía anteriormente, por lo que nos devuelve null.

```
            if (product == null = true) {   product: null
                MyUtils.print("\nProducto creado exitosamente");
            }
            else {
                MyUtils.print("\nProducto creado exitosamente");
                MyUtils.print("\nEl siguiente producto ha sido sobreescrito:");
                MyUtils.print(product.toString());
            }
            allSaved = false;
            MyUtils.pause();
            break;
```

Efectivamente, al tener devuelto null, significa que nos hemos deshecho del archivo dentro de storage, por lo que confirmamos que ha sido creado exitosamente.

```
            MyUtils.print("\nCreando un nuevo producto:");
            product = createProduct();
            product = storage.put(product.getCodigo(), product);   storage:  size = 7
            if (product == null) {
                MyUtils.print("\nProducto creado exitosamente");
            }
            else {
                MyUtils.print("\nProducto creado exitosamente");
                MyUtils.print("\nEl siguiente producto ha sido sobreescrito:");
                MyUtils.print(product.toString());   product: null
            }
            allSaved = false;   allSaved: false
            MyUtils.pause();
            break;
```

Se salta el else, y declaramos que la variable “necesitar cambios de confirmar”, que hemos llamado allSaved, se vuelve false, aunque ya era false.

```
101                 MyUtils.print("\nProducto creado exit
102                 MyUtils.print("\nEl siguiente producte
103                 MyUtils.print(product.toString());
104             }
105             allSaved = false;
106             MyUtils.pause();
107             break;
108         case '2':
109             MyUtils.print("\nMostrando todos los produ
```

Variables Console

Evaluate expression (Intro) or add a watch (Ctrl+Mayús+Intro)

options

The application is running

Y all llegar al pause() el programa continua ya que se necesita de input.

```
src
InputIncorrectoException
Main
MyUtils
Producto
.gitignore
Ficheros_UNICODE.iml
ternal Libraries
ratches and Consoles

        off – the start offset in the data
        len – the number of bytes that are written
    Throws: IOException – If an I/O error has occurred.
246 private native int readBytes(byte[] b, int off, int len) throws IOException;
247
    Reads up to b.length bytes of data from this input stream into an array of bytes. This method
    blocks until some input is available.
    Params: b – the buffer into which the data is read.
    Returns: the total number of bytes read into the buffer, or -1 if there is no more data because
             the end of the file has been reached.
    Throws: IOException – if an I/O error occurs.
    @Range(from = -1, to = java.lang.Integer.MAX_VALUE)
259 @Override
260 public int read(byte @notNull [] b) throws IOException {
261     long comp = Blocker.begin();
262     try {
263         return readBytes(b, off: 0, b.length);
264     } finally {
```

Main

Threads & Variables Console

"@1 in group "main": RUNNING

Evaluate expression (Intro) or add a watch (Ctrl+Mayús+Intro)

Bytes:-1, FileInputStream (java.io)

dden frames

e:22, MyUtils

options = Cannot find local variable 'options'

static members of FileInputStream

Variables debug info not available

Si le damos a pause, podemos ver que está leyendo bytes del scanner.

```
MyUtils
Producto
.gitignore
Ficheros_UNICODE.iml
External Libraries
Scratches and Consoles
```

```
210                 MyUtils.print("Ooops! No tendrias que estar aqui, in
211                 MyUtils.pause();
212                 break;
213             }
214         } while (selected != '7');
215         input.close();
216     }
217
218     private static String requestCodigo() {  3 usages
219         String codigo = "";
220         boolean success = false;
221         do {
222             try {
223                 MyUtils.printInLine(text: "\nIntroduce el codigo del pro
224                 codigo = MyUtils.inputRequest(PATTERNCOD);
225                 success = true;
226             } catch (InputIncorrectoException e) {
227                 MyUtils.print(e.getMessage());
228                 MyUtils.print("El codigo del producto tiene que ser 'P-X
```

```
ug   Main ×
Threads & Variables   Console
Introduce el nombre del producto: Espada

Introduce la cantidad del producto: 32

Introduce el precio del producto de hasta 2 decimales: 112,99

Producto creado exitosamente
========================================
Seleccione cualquier tecla para continuar
========================================
```

Le damos valores mientras esté pausado, y continuamos leyendo. Aunque ahora también vamos a usar este momento para realizar unas consultas a nuestro último objeto, ahora ya en storage.

```
s & Variables   Console
storage.get("P-0008").toString()
> path = ".\resources\"
```

Realizado.

```
torage.get("P-0008").toString()
result = "Producto{codigo='P-0008', nombre='Espada', cantidad=32, precio=112.99}"
> value = {byte[70]@1283} [80, 114, 111, 100, 117, 99, 116, 111, 123, 99, 111, 100, 105, 103, 111, 61, 39, 80, 45, 48, 48, 48, 56, 39, 44, 32, 110, 111, 109, 98, 114, 101, 61, 39, 69, 115, 112, 97, 100, 97, 39, 4 ... View
coder = 0
hash = 0
hashIsZero = false
```

```
storage.get("P-0008").setCantidad(100)
result = undefined
```

Podemos intentar probar a realizar cambios aquí. A través del debug.

```
storage = {LinkedHashMap@869}  size = 7
> "P-0001" -> {Producto@1299} "Producto{codigo='P-0001', nombre='Play Station 5', cantidad=2, precio=465434.54}"
> "P-0003" -> {Producto@1301} "Producto{codigo='P-0003', nombre='XBox 3', cantidad=43, precio=9000.0}"
> "P-0004" -> {Producto@1303} "Producto{codigo='P-0004', nombre='Palo de Pegar', cantidad=32, precio=1223.43}"
> "P-0005" -> {Producto@1305} "Producto{codigo='P-0005', nombre='Libros del Senior de los Anillos', cantidad=12, precio=200.0}"
> "P-0006" -> {Producto@1307} "Producto{codigo='P-0006', nombre='Marionetas', cantidad=24, precio=10.11}"
> "P-0007" -> {Producto@1309} "Producto{codigo='P-0007', nombre='Macarrones', cantidad=60, precio=5.5}"
> "P-0008" -> {Producto@1253} "Producto{codigo='P-0008', nombre='Espada', cantidad=100, precio=112.99}"
```

Y desde aquí abrimos storage y comprobamos.

```
"P-0008" -> {Producto@1253} "Producto{codigo='P-0008', nombre='Espada', cantidad=100, precio=112.99}"
  key = "P-0008"
  value = {Producto@1253} "Producto{codigo='P-0008', nombre='Espada', cantidad=100, precio=112.99}"
    codigo = "P-0008"
    nombre = "Espada"
    cantidad = 100
    precio = 112.99
```

Efectivamente, se ha guardado de forma correcta la cantidad usando el set dentro del propio debuggeador.

Con esto, se concluye el experimento.

Conclusiones

Con estos cambios, hemos logrado conseguir lo deseado. Que podamos leer lo que hace el programa.